Grokking as the Transition from Lazy to Rich Training Dynamics

Exercise Session - 1h, pen and paper

Reference: Kumar, Bordelon, Gershman & Pehlevan, ICLR 2024

Problem 1 - The Model and its Summary Statistics (10 min)

Consider a two-layer committee machine with N hidden neurons, input $\boldsymbol{x} \in \mathbb{R}^D$, and fixed readout weights all equal to 1:

$$f(\boldsymbol{w}, \boldsymbol{x}) = \frac{\alpha}{N} \sum_{i=1}^{N} \varphi(w_i \cdot \boldsymbol{x}), \qquad \varphi(h) = h + \frac{\epsilon}{2} h^2$$

The target function is $y(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{\beta}_\star \cdot \boldsymbol{x})^2$, with $\boldsymbol{x} \sim \mathcal{N}(0, \frac{1}{D}\boldsymbol{I})$.

(a) Define the two summary statistics:

$$\bar{\boldsymbol{w}} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{w}_i, \qquad \boldsymbol{M} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{w}_i \boldsymbol{w}_i^\top$$

Show that the network output can be written as:

$$f(\boldsymbol{x}) = \alpha\, \bar{\boldsymbol{w}} \cdot \boldsymbol{x} \; + \; \frac{\alpha\epsilon}{2}\, \boldsymbol{x}^\top \boldsymbol{M}\, \boldsymbol{x}$$

(b) What are the values of $\bar{\boldsymbol{w}}$ and $\boldsymbol{M}$ at random initialisation in the large N limit?

(c) For the network to perfectly fit the target on the test distribution, what must $\boldsymbol{M}$ and $\bar{\boldsymbol{w}}$ equal? (*Hint*: match the quadratic and linear parts of f to y.)

Solution

(a) Expand $\varphi(w_i \cdot x) = w_i \cdot x + \frac{\epsilon}{2}(w_i \cdot x)^2$. Sum over i:

$$f = \frac{\alpha}{N} \sum_i \left[w_i \cdot x + \frac{\epsilon}{2}(w_i \cdot x)^2 \right] = \alpha \left(\frac{1}{N} \sum_i w_i \right) \cdot x + \frac{\alpha\epsilon}{2}\, x^\top \left(\frac{1}{N} \sum_i w_i w_i^\top \right) x = \alpha\, \bar{w} \cdot x + \frac{\alpha\epsilon}{2}\, x^\top M\, x$$

(b) At random initialisation with i.i.d. w_i: by the law of large numbers, $\bar{\boldsymbol{w}} \to \mathbb{E}[w_i] = \boldsymbol{0}$ and $\boldsymbol{M} \to \mathbb{E}[w_i w_i^\top] = \boldsymbol{I}$ (assuming standard normal init with variance 1 per component).

(c) The target is $y = \frac{1}{2}(\beta_\star \cdot x)^2 = \frac{1}{2}x^\top \beta_\star \beta_\star^\top x$. Matching the quadratic part: $\frac{\alpha\epsilon}{2}M = \frac{1}{2}\beta_\star\beta_\star^\top$, so $\boldsymbol{M}^\star = \frac{1}{\alpha\epsilon}\boldsymbol{\beta}_\star\boldsymbol{\beta}_\star^\top$. The target has no linear component, so $\bar{\boldsymbol{w}}^\star = \boldsymbol{0}$.

Problem 2 - The NTK of the Toy Model (10 min)

(a) Compute the NTK $K(\boldsymbol{x}, \boldsymbol{x}') = \sum_{i=1}^N \nabla_{w_i} f \cdot \nabla_{w_i} f$ for this model. Show that it can be expressed in terms of $\bar{\boldsymbol{w}}$ and $\boldsymbol{M}$ as:

$$K(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}') + \epsilon\,(\boldsymbol{x} \cdot \boldsymbol{x}')\,\bar{\boldsymbol{w}} \cdot (\boldsymbol{x} + \boldsymbol{x}') + \epsilon^2\,(\boldsymbol{x} \cdot \boldsymbol{x}')\,\boldsymbol{x}^\top \boldsymbol{M}\,\boldsymbol{x}'$$

(*Hint*: compute $\nabla_{w_i} f$ first, then form the dot product.)

(b) At initialisation ($\bar{\boldsymbol{w}} = 0$, $\boldsymbol{M} = \boldsymbol{I}$), the kernel simplifies to $K_0(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}') + \epsilon^2(\boldsymbol{x} \cdot \boldsymbol{x}')^2$. This kernel is a sum of two terms. The first term $(\boldsymbol{x} \cdot \boldsymbol{x}')$ is sensitive to *linear* structure in the data, and the second $\epsilon^2(\boldsymbol{x} \cdot \boldsymbol{x}')^2$ to *quadratic* structure. Compute the ratio of the typical magnitude of the quadratic term to the linear term when x, x' are independent draws from $\mathcal{N}(0, \frac{1}{D}I)$. What does this tell you about how well-suited the initial kernel is for learning the (purely quadratic) target, especially when ϵ is small?

(*Hint*: use $\mathbb{E}[(\boldsymbol{x} \cdot \boldsymbol{x}')^2] = 1/D$ and $\mathbb{E}[(\boldsymbol{x} \cdot \boldsymbol{x}')^4] \sim 1/D^2$ for independent draws.)

(c) The target $y(x) = \frac{1}{2}(\beta_\star \cdot x)^2$ is purely quadratic. After feature learning, the network can align $\boldsymbol{M}$ with $\beta_\star\beta_\star^\top$, effectively reducing the problem to learning a single direction in $\mathbb{R}^D$. Argue intuitively that this requires $P \sim D$ samples. By contrast, a fixed kernel that treats all $\sim D^2$ quadratic directions equally would need far more samples. Why does create an opportunity for grokking?

Solution 2

(a) We have $\nabla_{w_i} f = \frac{\alpha}{N}\varphi'(w_i \cdot x)\,x = \frac{\alpha}{N}(1 + \epsilon\, w_i \cdot x)\,x$. Therefore:

$$K(x, x') = \sum_i \nabla_{w_i} f(x) \cdot \nabla_{w_i} f(x') = \frac{\alpha^2}{N^2}\sum_i (1 + \epsilon\, w_i \cdot x)(1 + \epsilon\, w_i \cdot x')\,(x \cdot x')$$

Expanding (and absorbing α^2/N into the normalisation by noting that the standard NTK definition for this parameterisation gives a factor that can be set to 1 with appropriate conventions):

$= (x \cdot x') + \epsilon(x \cdot x')\bar{w} \cdot (x + x') + \epsilon^2(x \cdot x')\,x^\top M\,x'$

(b) The linear term has typical magnitude $\mathbb{E}[|x \cdot x'|] \sim \sqrt{\mathbb{E}[(x \cdot x')^2]} = 1/\sqrt{D}$. The quadratic term has typical magnitude $\epsilon^2\,\mathbb{E}[(x \cdot x')^2] = \epsilon^2/D$. The ratio is:

$$\frac{\text{quadratic}}{\text{linear}} \sim \frac{\epsilon^2/D}{1/\sqrt{D}} = \frac{\epsilon^2}{\sqrt{D}}$$

For large D and fixed ϵ, this ratio is small: the initial kernel is dominated by its sensitivity to linear structure. Since the target is purely quadratic, the initial kernel is badly suited for the task. When ϵ is small, the mismatch is even worse.

(c) After feature learning, the network concentrates M onto the single direction $\beta_\star$. Identifying one direction in $\mathbb{R}^D$ requires $P \sim D$ samples (you need roughly D independent equations to pin down D unknowns). Without feature learning, the fixed kernel must resolve all $\sim D^2/2$ independent quadratic directions ($x_i x_j$) equally, since $M = I$ treats them all the same - this requires $P \sim D^2$ samples. The gap creates a *Goldilocks zone* $D \ll P \ll D^2$: enough data to generalise *if* features are learned, but not enough for the kernel method. A network that starts lazy will memorise via the kernel, then - once feature learning kicks in - will suddenly generalise. This delay is grokking.

Problem 4 - Loss Decomposition (15 min)

The test MSE of the model is $\mathcal{L} = \langle (y - f)^2 \rangle$ where $\langle \cdot \rangle$ is the expectation over $x \sim \mathcal{N}(0, \frac{1}{D} I)$.

Useful identity (Isserlis' theorem for Gaussians). For $x \sim \mathcal{N}(0, \frac{1}{D} I)$ and any symmetric matrices A, B:

$$\langle (x^\top A\, x)(x^\top B\, x) \rangle = \frac{1}{D^2} \left[(\operatorname{Tr} A)(\operatorname{Tr} B) + 2 \operatorname{Tr}(AB) \right]$$

In particular, $\langle (x^\top A\, x)^2 \rangle = \frac{1}{D^2} [(\operatorname{Tr} A)^2 + 2|A|_F^2]$ where $|A|_F^2 = \operatorname{Tr}(A^\top A)$.

(a) Decompose the network output as $f = f_{\text{lin}} + f_{\text{quad}}$ with $f_{\text{lin}} = \alpha \bar{\boldsymbol{w}} \cdot \boldsymbol{x}$ and $f_{\text{quad}} = \frac{\alpha\epsilon}{2} \boldsymbol{x}^\top \boldsymbol{M}\, \boldsymbol{x}$. Note that the target $y = \frac{1}{2} \boldsymbol{x}^\top \boldsymbol{\beta}_\star \boldsymbol{\beta}_\star^\top \boldsymbol{x}$ is purely quadratic. Use the fact that odd moments of isotropic Gaussians vanish (i.e. $\langle x_i\, g(x^\top A x) \rangle = 0$) to argue that the MSE splits as:

$$\mathcal{L} = \langle (y - f_{\text{quad}})^2 \rangle + \langle f_{\text{lin}}^2 \rangle$$

Then compute $\langle f_{\text{lin}}^2 \rangle$ explicitly. This is **Term C** (linear power).

(b) Now focus on the quadratic part. Define $A = \boldsymbol{\beta}_\star \boldsymbol{\beta}_\star^\top - \alpha\epsilon \boldsymbol{M}$, so that $y - f_{\text{quad}} = \frac{1}{2} \boldsymbol{x}^\top A\, \boldsymbol{x}$. Apply the Isserlis identity above to compute $\langle (y - f_{\text{quad}})^2 \rangle = \frac{1}{4} \langle (\boldsymbol{x}^\top A\, \boldsymbol{x})^2 \rangle$. Show that the result can be written as the sum of two terms:

$$\underbrace{\frac{1}{4D^2}(\operatorname{Tr} A)^2}_{\text{Term A: variance error}} \quad + \quad \underbrace{\frac{1}{2D^2}|A|_F^2}_{\text{Term B: misalignment error}}$$

Write out Term A and Term B explicitly in terms of $|\boldsymbol{\beta}_\star|^2$, $\operatorname{Tr}\boldsymbol{M}$, α, ϵ, and $|\alpha\epsilon\boldsymbol{M} - \boldsymbol{\beta}_\star\boldsymbol{\beta}_\star^\top|_F$.

(c) Interpret each of the three terms (A, B, C) in one sentence.

(d) At initialisation ($\bar{\boldsymbol{w}} = 0$, $\boldsymbol{M} = \boldsymbol{I}$), which terms are large and which are small? What happens in early training: recalling from Problem 3(b) that the initial kernel is much more sensitive to linear structure than quadratic structure, which component of f does the network use first to reduce train loss, and what does this do to the three terms?

Solution 4

(a) We have $y - f = (y - f_{\text{quad}}) - f_{\text{lin}}$. Expanding the square:

$$\mathcal{L} = \langle (y - f_{\text{quad}})^2 \rangle - 2\langle (y - f_{\text{quad}})\, f_{\text{lin}} \rangle + \langle f_{\text{lin}}^2 \rangle$$

The cross-term vanishes: $f_{\text{lin}} = \alpha\bar{w} \cdot x$ is a linear function of x, while $y - f_{\text{quad}}$ is a quadratic form $x^\top(\cdots)x$. Their product is a degree-3 polynomial in x, and all odd-degree moments of an isotropic Gaussian are zero. Therefore $\mathcal{L} = \langle (y - f_{\text{quad}})^2 \rangle + \langle f_{\text{lin}}^2 \rangle$.

Term C: $\langle f_{\text{lin}}^2 \rangle = \alpha^2 \langle (\bar{w} \cdot x)^2 \rangle = \alpha^2 \sum_{ij} \bar{w}_i \bar{w}_j \langle x_i x_j \rangle = \alpha^2 \sum_i \bar{w}_i^2 \cdot \frac{1}{D} = \frac{\alpha^2}{D}|\bar{\boldsymbol{w}}|^2$.

(b) We have $y - f_{\text{quad}} = \frac{1}{2}x^\top A\, x$ with $A = \beta_\star\beta_\star^\top - \alpha\epsilon M$. So:

$$\langle (y - f_{\text{quad}})^2 \rangle = \frac{1}{4}\langle (x^\top A\, x)^2 \rangle = \frac{1}{4D^2}\left[(\operatorname{Tr} A)^2 + 2|A|_F^2\right]$$

This gives:

$$\text{Term A} = \frac{1}{4D^2}(\operatorname{Tr} A)^2 = \frac{1}{4}\left(\frac{|\beta_\star|^2 - \alpha\epsilon \operatorname{Tr} M}{D}\right)^2$$

$$\text{Term B} = \frac{1}{2D^2}|A|_F^2 = \frac{1}{2D^2}|\alpha\epsilon\, M - \beta_\star\beta_\star^\top|_F^2$$

(c) - **Term A (variance error)**: Measures whether the overall *scale* of the learned quadratic form ($\alpha\epsilon \operatorname{Tr} M$) matches the target's scale ($|\beta_\star|^2$). - **Term B (misalignment error)**: Measures whether the *shape* (directional structure) of M is aligned with the rank-1 target $\beta_\star\beta_\star^\top$ - this is the term most closely related

to NTK alignment / CKA. - **Term C (linear power)**: Penalises any energy the network puts into linear functions of the input, since the target is purely quadratic and contains no linear component.

(d) At initialisation, $\bar{w} = 0$ and $M = I$: - **Term C = 0** (no linear power yet). - **Term A** depends on $(|\beta_\star|^2 - \alpha\epsilon D)^2/D^2$, which is nonzero - **Term B is large**: the identity matrix I is spread uniformly across all D directions, while $\beta_\star\beta_\star^\top$ is rank-1. The Frobenius distance $|\alpha\epsilon I - \beta_\star\beta_\star^\top|_F$ is large.

In early training, the network needs to reduce train loss. From Problem 3(b), the initial kernel's sensitivity to linear structure (scaling as $1/\sqrt{D}$) is much stronger than its sensitivity to quadratic structure (scaling as ϵ^2/D). So the network first exploits the linear component of the activation $\varphi(h) = h + \frac{\epsilon}{2}h^2$: it grows $\bar{w}$ to produce a linear function $\alpha\bar{w} \cdot x$ that approximately fits the training data. This *reduces train loss* but *increases Term C* on the test set - the network is fitting a linear function to a quadratic target, which works on finite training data but fails to generalise. Meanwhile Term B (misalignment) remains large because no feature learning has occurred. This is the memorisation phase.

Problem 5 - The Role of α: Controlling Laziness (10 min)

(a) Consider training with gradient descent on MSE loss $\mathcal{L} = \frac{1}{2}\sum_\mu (y_\mu - f(\boldsymbol{w}, \boldsymbol{x}_\mu))^2$. In the lazy regime, f is linear in $\boldsymbol{w}$, so the gradient descent dynamics become $\dot{\boldsymbol{w}} = -\nabla_w \mathcal{L}$. For a linear model $f_\mu \approx f_\mu^0 + \boldsymbol{J}_\mu \cdot (\boldsymbol{w} - \boldsymbol{w}_0)$ where $\boldsymbol{J}_\mu = \nabla_w f|_{w_0}(\boldsymbol{x}_\mu)$, the converged solution satisfies $f_\mu = y_\mu$ for all training points. Show that the required parameter displacement is:

$$\boldsymbol{w}^* - \boldsymbol{w}_0 = \boldsymbol{J}^\top (K_0)^{-1}(\boldsymbol{y} - \boldsymbol{f}^0)$$

where $(K_0)_{\mu\nu} = \boldsymbol{J}_\mu \cdot \boldsymbol{J}_\nu$ is the NTK Gram matrix. Deduce that $|\boldsymbol{w}^* - \boldsymbol{w}_0|^2 = (\boldsymbol{y} - \boldsymbol{f}^0)^\top K_0^{-1}(\boldsymbol{y} - \boldsymbol{f}^0)$.

(*Hint*: the converged condition is $\boldsymbol{J}_\mu \cdot (\boldsymbol{w}^* - \boldsymbol{w}_0) = y_\mu - f_\mu^0$ for all μ. This is a linear system.)

(b) Now recall that the network output is $f = \frac{\alpha}{N}\sum_i \varphi(w_i \cdot x)$. A small parameter change δw produces a change in the output $\delta f \sim \alpha\,|\delta w|$ (because the Jacobian $\nabla_w f$ scales as α). Since the target is $O(1)$, the network needs $\delta f \sim O(1)$ to fit the data. Use this to argue that $|\delta w| \sim 1/\alpha$. Why does this mean that large α keeps the network in the lazy regime?

(c) Feature learning corresponds to the *nonlinear* part of the dynamics - the deviation of $f(\boldsymbol{w})$ from its linearisation. One can show that the rate at which the NTK itself changes (i.e. the rate at which features evolve) scales as $\dot{K}/K \sim |\delta w|^2 \sim 1/\alpha^2$. Meanwhile, the kernel regression solution is reached on a timescale that does not grow with α. This creates a separation of timescales.

On the diagram below, fill in the three labels (i), (ii), (iii) and mark the grokking gap. Then sketch a second curve (dashed) showing what happens when α is *increased*.

```
Loss
 |
 |  -----------
 | /            \-----------
 |/                         \
 |                           \-------------
 |
 +------+----------+-----------+-----------> log(time)
       (i)        (ii)       (iii)

 (i)   = ...................
 (ii)  = ...................
 (iii) = ...................

 Grokking gap = from ......... to .........
```

(*Hint*: one curve is train loss, one is test loss. They start together, then diverge, then reconverge.)

(d) What happens in the limits $\alpha \to 0$ and $\alpha \to \infty$?

Solution 5

(a) The interpolation condition $J_\mu \cdot (\boldsymbol{w}^* - \boldsymbol{w}_0) = y_\mu - f^0_\mu$ for all μ can be written in matrix form as $\boldsymbol{J}(\boldsymbol{w}^* - \boldsymbol{w}_0) = \boldsymbol{y} - \boldsymbol{f}^0$, where $\boldsymbol{J}$ is the $P \times \dim(\boldsymbol{w})$ Jacobian matrix. The minimum-norm solution is $\boldsymbol{w}^* - \boldsymbol{w}_0 = \boldsymbol{J}^\top(\boldsymbol{J}\boldsymbol{J}^\top)^{-1}(\boldsymbol{y} - \boldsymbol{f}^0) = \boldsymbol{J}^\top K_0^{-1}(\boldsymbol{y} - \boldsymbol{f}^0)$, since $K_0 = \boldsymbol{J}\boldsymbol{J}^\top$. Then:

$$|\boldsymbol{w}^* - \boldsymbol{w}_0|^2 = (\boldsymbol{y} - \boldsymbol{f}^0)^\top K_0^{-1}\boldsymbol{J}\boldsymbol{J}^\top K_0^{-1}(\boldsymbol{y} - \boldsymbol{f}^0) = (\boldsymbol{y} - \boldsymbol{f}^0)^\top K_0^{-1}(\boldsymbol{y} - \boldsymbol{f}^0)$$

(b) Since $\delta f \sim \alpha\,|\delta w|$ and δf must be $O(1)$ to fit targets of order 1, we need $|\delta w| \sim 1/\alpha$. Large α means tiny parameter changes suffice to fit the data. When parameters barely move, the Taylor expansion $f(w) \approx f(w_0) + \nabla f|_{w_0} \cdot (w - w_0)$ remains accurate - the network stays in the linearised/lazy regime. The features $\nabla_w f|_w$ barely change from their initial values.

(c) Completed diagram:

```
Loss
 |
 |  train ___________                          test
```

```
| /                 \___________              (solid = small alpha,
|/                              \               dashed = large alpha)
|                                \_____________
+------+--------------+-----------+-----------> log(time)
      (i)           (ii)        (iii)

(i)   = train loss reaches zero (kernel regime fits training data)
(ii)  = feature learning onset (NTK begins to rotate, M starts aligning with beta_*)
(iii) = test loss drops (generalisation achieved)

Grokking gap = from (i) to (iii)
```

When α is increased, (i) stays roughly fixed (kernel fitting speed doesn't depend much on α), but (ii) and (iii) shift to the right (feature learning rate $\sim 1/\alpha^2$ is slower). The grokking gap widens.

(d) - $\alpha \to 0$: Feature learning is immediate. Train and test loss decrease together - **no grokking**. The network goes directly to the rich/feature-learning solution. - $\alpha \to \infty$: The network is permanently lazy. It converges to the kernel regression solution with the initial NTK, which is misaligned. Train loss can reach zero on finite data, but test loss plateaus at the (poor) kernel regression error - **grokking never completes**.

Problem 6 - The Role of ϵ: Initial Kernel Alignment (5 min)

(a) From Problem 3, the initial kernel eigenvalue on quadratic functions is $\lambda_{\text{quad}} = 2\epsilon^2/D^2$. Define the *NTK alignment* $\varepsilon = \boldsymbol{y}^\top K_0 \boldsymbol{y}/(\|K_0\|_F \|\boldsymbol{y}\|^2)$. How does this alignment scale with ϵ in the toy model?

(b) Large ϵ means the initial kernel already places significant power on quadratic functions. Explain why this *reduces* or *eliminates* grokking. Conversely, why does small ϵ *increase* the grokking delay but lead to *lower* final test loss?

Solution 6

(a) Since $\lambda_{\text{quad}} \propto \epsilon^2$ and the target is purely quadratic, the kernel-target alignment scales as $\varepsilon \propto \epsilon^2$. Larger ϵ means the initial kernel is better matched to the target.

(b) Large ϵ: recall from Problem 3(b) that the initial kernel's sensitivity to quadratic structure scales as ϵ^2/D. When ϵ is large, this quadratic sensitivity is comparable to the linear sensitivity ($1/\sqrt{D}$), so the initial kernel already "sees" the quadratic target almost as well as it sees linear functions. This means kernel regression at initialisation can fit the target reasonably well without needing to change features - the network generalises early and there is no grokking gap.

Small ϵ: the initial kernel is nearly blind to quadratic structure (its quadratic sensitivity $\epsilon^2/D \ll 1/\sqrt{D}$). The kernel regression solution at initialisation fits the training data primarily through linear components (growing $\bar{w}$, increasing Term C), which fail to generalise. The network *must* eventually learn features (rotate M toward $\beta_\star\beta_\star^\top$) to generalise, and this takes time - producing a long grokking delay. However, once features are learned, the network finds a solution specialised to the target, which can achieve lower test error than the generic kernel solution. Bad initial features force more feature learning, resulting in a better final solution.

Problem 7 - Synthesis: Why Grokking is a Lazy-to-Rich Transition (5 min)

(a) State the three conditions identified by Kumar et al. that are jointly sufficient for grokking.

(b) Using the loss decomposition from Problem 4, describe the two-phase dynamics during grokking: - Phase 1 (early training): What happens to Terms A, B, C? - Phase 2 (late training): What happens to Terms A, B, C?

(c) Explain in 2-3 sentences why weight decay is *sufficient but not necessary* for grokking, and how the lazy-to-rich framework subsumes weight-decay-based explanations.

Solution 7

(a) The three conditions are: 1. The initial NTK is *misaligned* with the target (small ϵ, or low CKA). 2. The dataset size is in a *Goldilocks zone*: large enough for generalisation to be possible, but small enough that train and test loss don't track each other. 3. The network starts in the *lazy regime* (large α), so feature learning is delayed.

(b) - **Phase 1** (memorisation): The network uses the linear component of φ to approximately fit the training data via kernel regression. Term C increases (linear power grows), Term B stays large (no alignment), Term A may fluctuate. Train loss decreases, but test loss stays high or even increases. - **Phase 2** (generalisation/grokking): Feature learning kicks in. M rotates toward $\beta_\star\beta_\star^\top$ (Term B decreases), $\bar{w}$ is driven back to zero (Term C decreases), and Term A adjusts as $\mathrm{Tr}M$ calibrates. Test loss finally drops.

(c) Weight decay shrinks the parameter norm, which eventually pushes the network out of the lazy regime (since small weights make the linearisation less dominant), triggering feature learning. But any mechanism that delays the transition from lazy to rich dynamics - such as a large output scale α - produces the same effect without weight decay. The lazy-to-rich framework is thus more

fundamental: weight decay is one of several ways to control the transition speed, not the root cause.